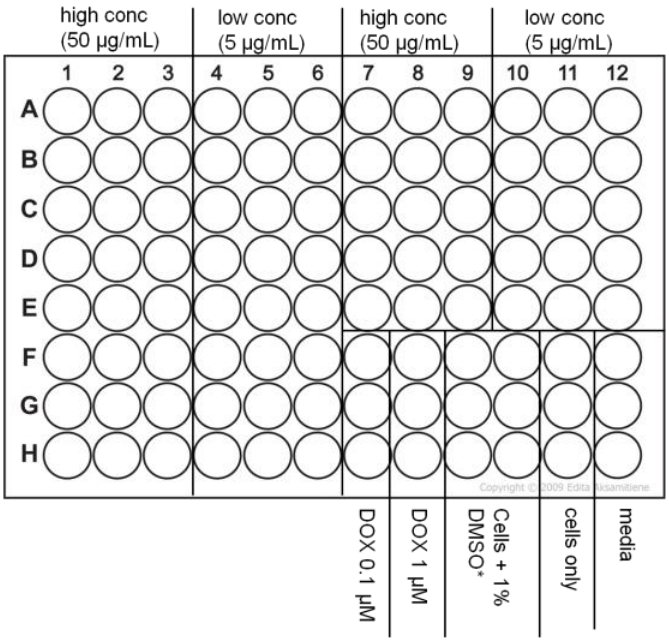
Final Project Bio 539

Peran

Microdilution-based antimicrobial assay is one of the common screening tools in the laboratory. This high-throughput assay allows testing of more samples compared to traditional disc diffusion assay. Using redox dye such as resazurin or MTT reagent, mitochondrial enzyme activity can be quantified as a measure of bacterial metabolism or growth. This method is also versatile to various pathogens such as the ESKAPE which are deemed multi-drug-resistant bacteria.

This is usually performed in parallel with chemistry fractionations to determine which samples/ fractions are bioactive until we obtain the pure bioactive compound. The usual screening assay template is shown below.



Traditionally, fluorescence readings were processed in Microsoft Excel and then exported to GraphPad Prism for visualization. The computation of the % bacterial inhibition is shown below. This computation highlights the importance of the wells corresponding to culture media (sterility) and growth control (negative control).

$$\%\ inhibition\ = \left(1 - \frac{RFU\ sample - RFU\ sterility\ control}{RFU\ negative\ control - RFU\ sterility\ control}\right) x100$$

However, because the wells assigned to media and growth controls can vary between experiments, their positions are not always consistent with a fixed template. This makes Excel-based analysis prone to errors and time-consuming to adjust. To address this, I developed a Python-based workflow that allows flexible tagging of control wells. Using this method will allow us to determine the bioactive fractions faster.

Our new method utilized python analysis which allows a simple command via terminal. The run command will only contain the following: the script, metadata file, filenames of the raw data (in csv), and the well locations for media and growth controls. Sample usage is shown below.

```
#!/usr/bin/env python3
# usage: python inhibition.py <metadata.csv> <filename1> <filename2> <filename...n> <media control wells> <growth control wells>
# usage example: python inhibition.py metadata.csv plate1.csv plate2.csv plate3.csv plate4.csv H10,H11,H12 H7,H8,H9

```

This correlates to the loading of the data using the following arguments. Using the script below will allow future users to add indefinite number of raw files. The only consideration is that the raw files must came from one batch containing same location of control wells (growth and media). Meanwhile, the metadata added here would be used for plotting later, so that information such as doses, sample codes, etc can be incorporated during the summary files and plots. The metadata also prevents hardcoding, so users can add more columns on metadata and tweak some of the codes if they prefer.

```
# LOADING OF DATA VIA ARGUMENTS
if len(sys.argv) < 5:
                    #0-python script i.e. script.py
                    #1-filename1
                    #2-filename2
                    #3-metadata
                    #4-media control
                    #5-growth control
    sys.exit("Usage: python y12.py metadata.csv plate1.csv plate2.csv ... MEDIA_WELLS GROWTH_WELLS")

#This is similar to $1 in Shell scripting where we can incorporate the filename to our phython command in the terminal.
#But in this case, we added not just the filenames, but the metadata, and the wells for growth and media control
#The following commands identifies which argument is the metadata, media blanks, and growth
metadata_file = sys.argv[1]
media_arg = sys.argv[-2]
growth_arg = sys.argv[-1]

#This easily identifies our files so that even if we add more files on the middle, we dont have to edit the script.
#Hence the file_list are all from index 2 until before the last two argument.
file_list = sys.argv[2:-2]

if len(file_list) == 0:
    sys.exit("ERROR: No plate files provided.") #just a loading error check if no files are provided

```

Next, we converted the raw data into usable string coordinates so that the fluorescence value of each well can be recognized later. Most importantly, the values of media blanks and growth control were identified. This is essential in the calculation of our % bacterial inhibition.

```
#LOADING RAW DATA
#Converts the well location into usable string coordinates (a1,a2,a3...h12) --> [("a,1"), ("a,2"), ("a,3")]
def parse_wells(s):
    return [(w[0], int(w[1:])) for w in s.split(",")]

#get the media_blank and growth values based on the parsed values
MEDIA_BLANK = parse_wells(media_arg)
GROWTH_CONTROL = parse_wells(growth_arg)

```

Then, the average of media and growth control per plate was then computed. The mean of the controls was derived from each plate since the plate might have unique conditions from each. Here, I used NumPy package to avoid writing extensive loop function – especially that the mean of the controls will be used repeatedly to compute the % inhibition in every wells. So, using numpy would be more efficient in data analysis especially if having many raw files. After this, we can now compute for the % bacterial inhibition as shown below.

```python
# COMPUTATION OF OUR MEDIA (STERILITY) and GROWTH (negative) CONTROLS
#-------------------------------------------------
#Here we used the numpy to have faster math operations and to avoid extensive scripts (if loop is used)
#First we get the mean values of the media and growth for each plate. The values are stored as numpy arrays here
def compute_controls(plate):
    media_vals = np.array([plate.loc[r, c] for r, c in MEDIA_BLANK])
    growth_vals = np.array([plate.loc[r, c] for r, c in GROWTH_CONTROL])

    return np.mean(media_vals), np.mean(growth_vals) #this computes the average/mean for both controls.



# COMPUTES FOR % BACTERIAL INHIBITION
#-------------------------------------------------
#This formula here can also be changed if other analysis will be needed
#(i.e. %bacterial growth proliferation which is just the inverse of the % inhibition)

def compute_inhibition(plate, media_avg, growth_avg):
    values = plate.values #creates numpy values from each well the samples
    inhibition = (1 - ((values - media_avg) / (growth_avg - media_avg))) * 100 #formula for getting the %inhibition on EACH WELL

    return pd.DataFrame(inhibition, index=plate.index, columns=plate.columns)
```

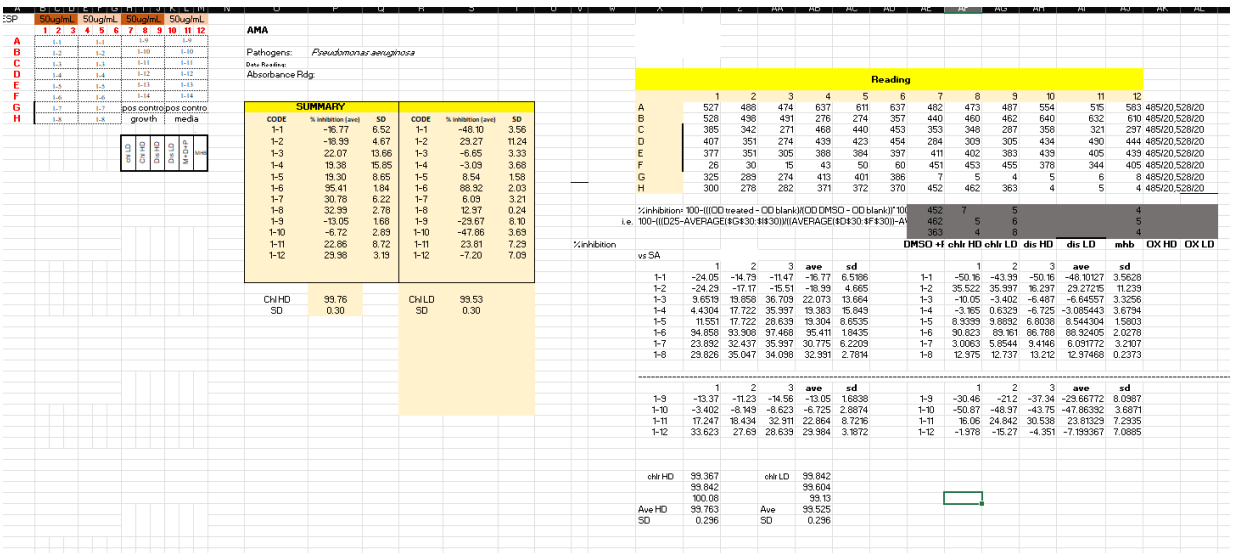
At this point, the % inhibition values will be computed on each well. But since the samples are replicates (n=3), we now have to group together our similar samples to compute for mean % inhibition. This is where the metadata file is important since it shows how our samples are grouped together (i.e. a1, a2, and a3 are similar sample). For this part, I asked chatgpt how to merge the metadata with our % inhibition values. It suggested to convert my pandas dataframe into long table format as shown below to solve the merging issue since our data above are stored as pandas array.

```python
# NOTE: This % inhibition values are computed per well. Later we want to summarize them per sample
# as mean_inhibition with se (standard error), which will be summarized later in the output file "combined_summary.csv"
# Our intended output file contains the ff info: plate,sample_code,dose,mean_inhibition,se
# As you can see, some of the details included here (i.e. sample_code, dose) must be taken from the metadata.csv.
# Hence, we have to prepare our dataset into a compatible format for merging

# CONVERTING PANDAS DATAFRAME (containing Numpy array) INTO LONG TABLE FORMAT - suggested by chatgpt
#-------------------------------------------------
# This step is similar to data preparation in Rscript prior to joining metadata to our dataframe.
# We need this step to be able to merge data from Metadata.csv later.
# This converts numpy backed dataframe into long table format which is compatible later during
#the "df.merge(metadata, on=["row", "col"])

def plate_to_long(inhibition_plate):
    return (
        inhibition_plate
        .reset_index()
        .melt(id_vars="index", var_name="col", value_name="inhibition")
        .rename(columns={"index": "row"})
    )
```

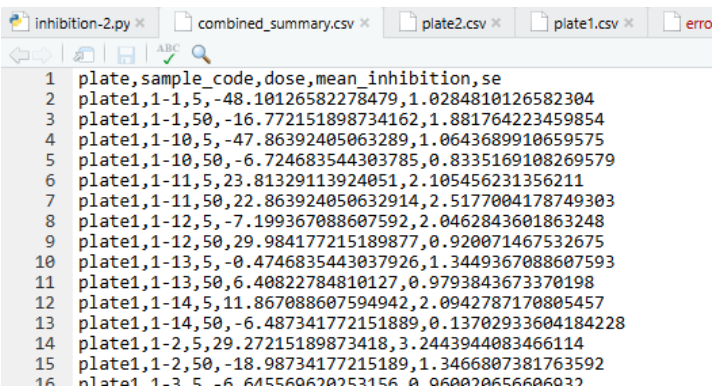
Once converted, the mean % inhibition per sample (n=3) can be computed as well as the standard error. The merging and correct grouping is very important so that the averaged values are correct.

```python
# GROUPS AND SUMMARIZE THE % INHIBITION VALUES
#-----------------------------------------------
#Note: Our % inhibition values above are computed per well. But since our setup is organized in triplicate values, we want to
#obtain the mean % inhibition as well as the standard error to reveal the variation in our sample.

def compute_summary(long_df, plate_name):
    df = long_df.merge(metadata, on=["row", "col"]) #this now merge the data "long_df"" which contains the inhibition values,
                                                    # and "metadata" which contains the row, col, sample_code, and dose
    df["plate"] = plate_name                        #plate stored as plate_name

    df = df.dropna(subset=["inhibition"])           #remove the row if NaN is observed

    grouped = df.groupby(["plate", "sample_code", "dose"])["inhibition"]    #This creates our sample grouping

                            #Meaning a1, a2, and a3 from Plate1 having same dose is considered as triplicates having same condition
                            #this is important since we want to get the mean %inhibition per triplicate of the samplee
    return grouped.agg(
        mean_inhibition="mean",
        se=lambda x: x.std(ddof=1) / np.sqrt(len(x)) #computes for standard error
    ).reset_index()
```

To validate if the mean inhibition was calculated correctly in our script, we used our traditional MS Excel computation which showed parallel values. Sample MS Excel for plate1.csv analysis is showed below. Traditionally, we organize the mean % inhibition from our excel and transport them to Graphpad Prism to plots our bar graph and heatmaps.



The traditional approach becomes inefficient when handling many plates, since it relies heavily on repeated copy-and-paste steps and increases the risk of human error, which can lead to unreliable plots. It also requires managing multiple spreadsheets and manually consolidating them into a single summary table. This new python script addresses that issue by automatically generating a combined_summary.csv file, which compiles the average % inhibition for each sample across plates, as shown below.



Aside from this we also added some extra filtering of dataset, which can reveal the list of samples having high (>60%) inhibition. This easily identifies which samples are bioactive and worth pursuing. The active_hits.csv filters summarize the active antimicrobial samples. Other users can also change their percentage here if they preferred. This section was made with the help of chatgpt.

```python

# OUTPUT SUMMARY FILE
#-----------------------------------------------

all_summary = [] # this contains the list for the summary per plate
all_inhibition = [] #creates list for the inhibition which contains plate_name, inhibition_matrix

for FILE_PATH in file_list: #go through each plate

    plate_name = os.path.splitext(os.path.basename(FILE_PATH))[0] #extract the plate name
                                                                  #(i.e. /data/plate1.csv will retrieve just the plate1.csv)
    plate = load_plate(FILE_PATH) #loads the folder path where the file is located
    media_avg, growth_avg = compute_controls(plate) #then compute for the controls per plate
    inhibition_plate = compute_inhibition(plate, media_avg, growth_avg) #proceed with computing the %inhibition per well
    all_inhibition.append((plate_name, inhibition_plate)) #store the inhibition data into inhibition_plate.
                                                          #This will be used for heatmap
    long_df = plate_to_long(inhibition_plate) #converts to compatible long format (i.e well a1 to row a and column 1)
    summary = compute_summary(long_df, plate_name) #now let;s merge the metadata with the compute_summary
    all_summary.append(summary) #and store it as summary

BASE_DIR = os.path.dirname(file_list[0]) #setting our output directory
combined_summary = pd.concat(all_summary, ignore_index=True)
combined_summary.to_csv(os.path.join(BASE_DIR, "combined_summary.csv"), index=False)

#This filters out samples having >60% inhibition. this is helpful to have a summarized list of samples
# which we can consider as active antimicrobial agent.
active_hits = combined_summary[combined_summary["mean_inhibition"] > 60]
active_hits.to_csv(os.path.join(BASE_DIR, "active_hits.csv"), index=False)
```

At the same time, I incorporated several safeguards into the analysis. While this is not implemented using pytest, it is designed to reflect the types of errors discussed in class. With assistance from ChatGPT, the script filters out problematic raw data, including wells with OVERFLOW values (indicating maxed-out fluorescence readings), negative values, and missing/ blank entries. Rather than serving as a formal testing framework, this approach helps flag questionable raw data so users can revisit and verify the accuracy of their raw inputs. This section was actually placed on top portion of the script, after the loading of the raw data.

```python
# This section checks if my raw data values are okay for processing. This reflects real-life cases where
# the raw data might contain possible errors. For example, the most common issues in reading plates would
# be a sudden "OVERFLOW"  when the reader's capacity has been maxed out. Possible human error could also occur
# happen in acquiring the data such as missing, negative, or letter-based values which are deemed erroneous.
# This section allows raw data values to be converted into a float ONLY IF our values are valid.
#I asked chatgpt for a script to log these potential errors, AND if observed, tabulate them into error_output.csv
# This allows to easily trace the location of wells having errors, so that users can re-evaluate the raw files.

    for r in df.index:
        for c in df.columns: #Go through every column and row

            val = df.loc[r, c] #get the values for each well [r, c] and check for the ff:

            #1. MISSING VALUES
            if pd.isna(val) or str(val).strip() == "": #== "" empty/ blank cells
                ALL_ERRORS.append({                    #if present, log/ append the error
                    "plate": plate_name,               #log the plate number/name
                    "sample_code": f"{r}{c}",          #log the sample_code
                    "error": "missing_value"           #and write missing_value as the type of error
                })
                cleaned.loc[r, c] = np.nan
                continue
            val_str = str(val).strip()

            #2. NUMERIC WITH COMMAS
            if re.match(r'^-?\d{1,3}(,\d{3})*(\.\d+)?$', val_str): #this detects 1,234 and replace it with 1234.

                num = float(val_str.replace(',', ''))
                if num < 0:                                #This also logs negative values as error
                    ALL_ERRORS.append({                    #Same thing above, which logs the information about the errors for traceability
                        "plate": plate_name,
                        "sample_code": f"{r}{c}",
                        "error": "neg_value_raw"            #log/append the error as neg_value_raw in our error_output.csv
                    })
                    cleaned.loc[r, c] = np.nan
                    continue
                cleaned.loc[r, c] = num
                continue

            #3. INVALID STRING (OVERFLOW, abc, etc.)
            ALL_ERRORS.append({
                "plate": plate_name,
                "sample_code": f"{r}{c}",
                "error": val_str
            })
            cleaned.loc[r, c] = np.nan #any error will be removed from calculations and will be considered NaN automatically
    # Final numeric conversion safety net
    cleaned = cleaned.astype(float) #cleaned dataset contains all the valid numbers stored as float
```

The code above lets us identify these errors which can be now exported as error_output.csv if errors are detected. To show that this is working, I deliberately made some errors on Plate4.csv to see if it will be detected. And yes, it was successfully detected by our script.

```csv
plate,sample_code,error
plate4,A2,OVERFLOW
plate4,D6,neg_value_raw
plate4,E8,neg_value_raw
plate4,G1,missing_value
```

Lastly, with the introduction of combined_summary.csv, which consolidates the % inhibition data across all plates, we can now generate visualizations using heatmaps and bar plots. Initially, the plan was to use R with ggplot2 for plotting, but after discussing Matplotlib in class, I opted to implement the visualizations in Python with assistance from ChatGPT. This allows for a single streamlined workflow that handles both analysis and visualization in one step for bioactivity assessment. For further details on each plot, please refer to the comments within the script (inhibition.py).

Using the methods above, we were able to obtain the following plots from my raw data. Kindly refer to next page

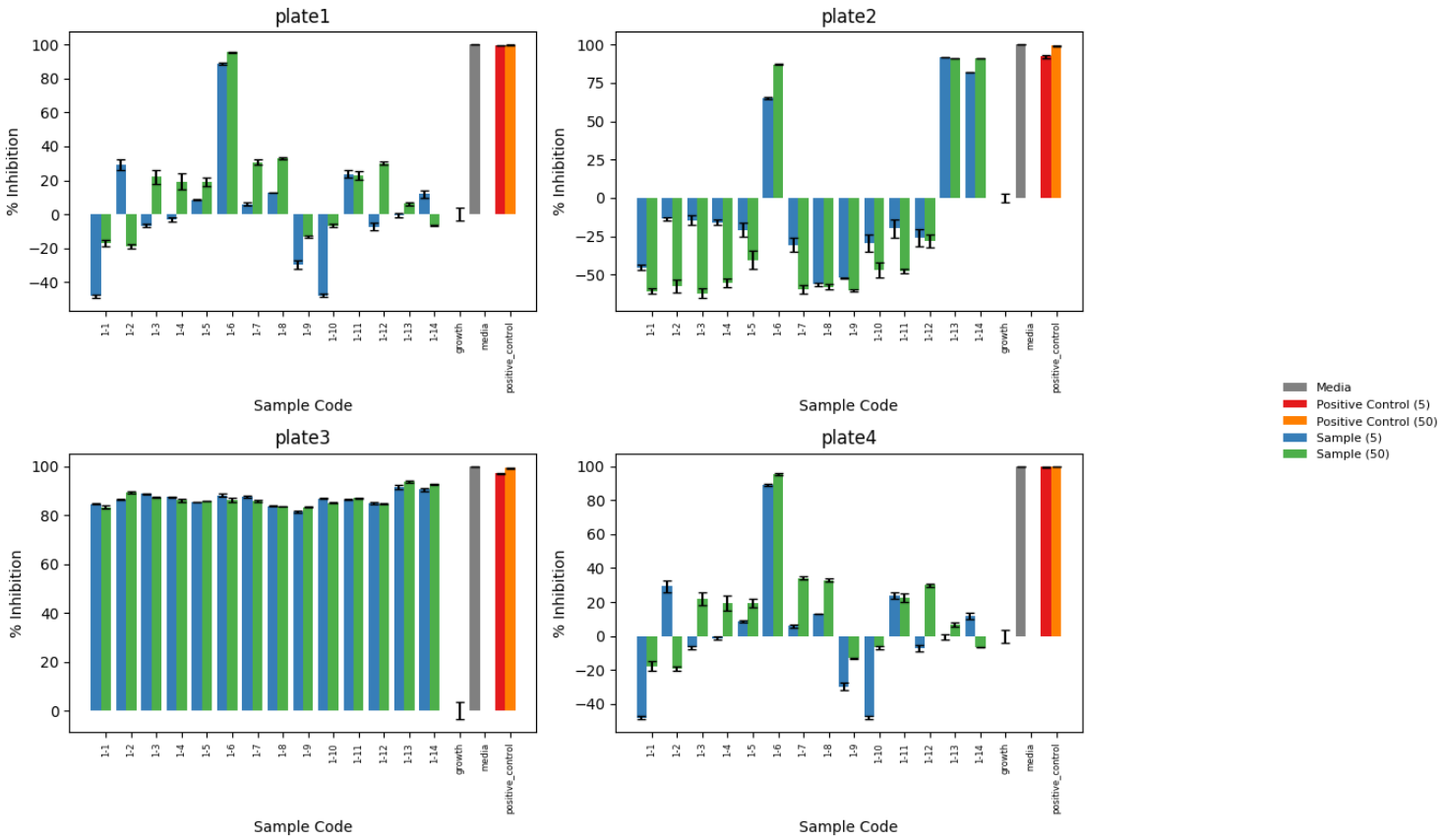
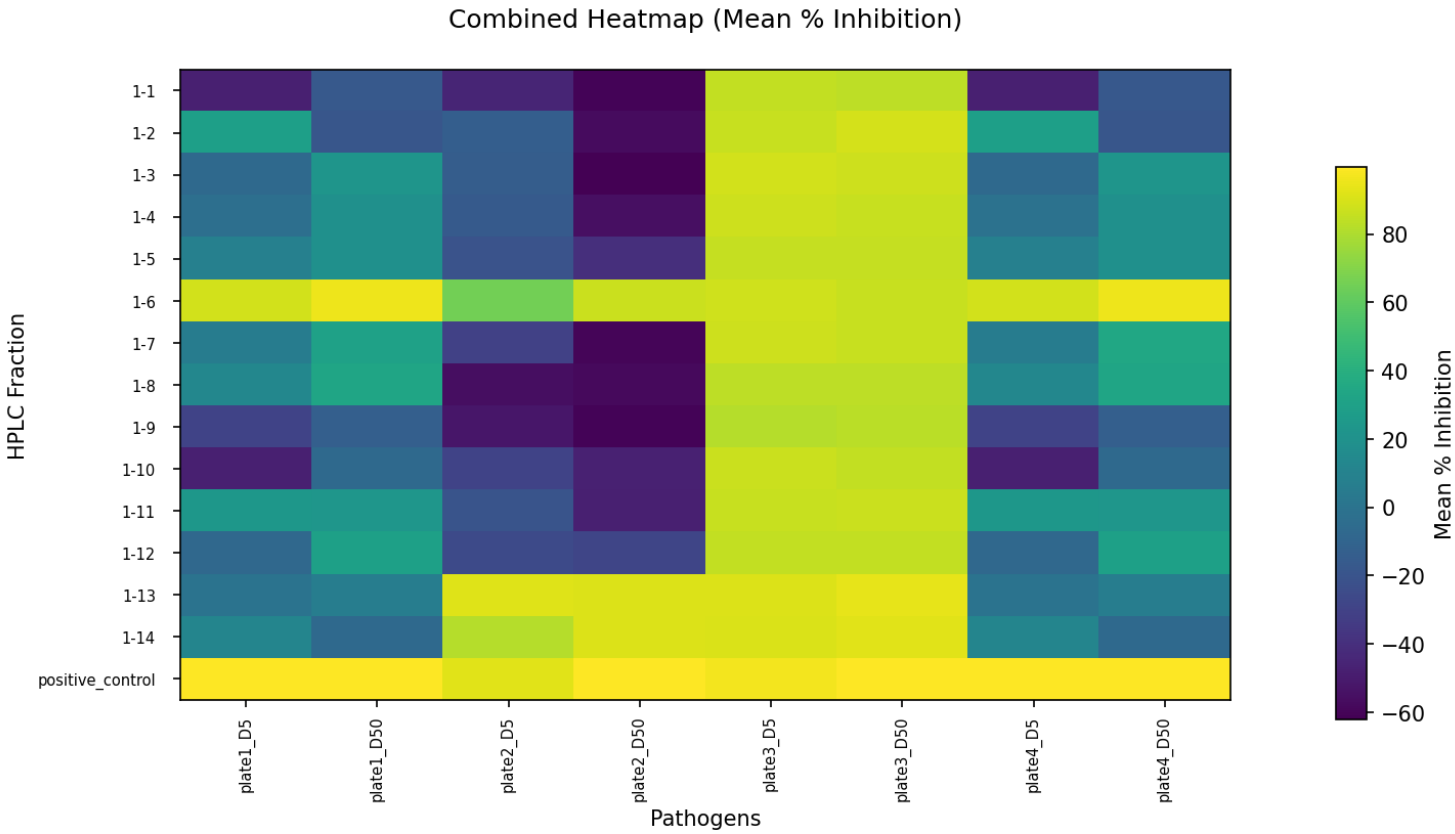
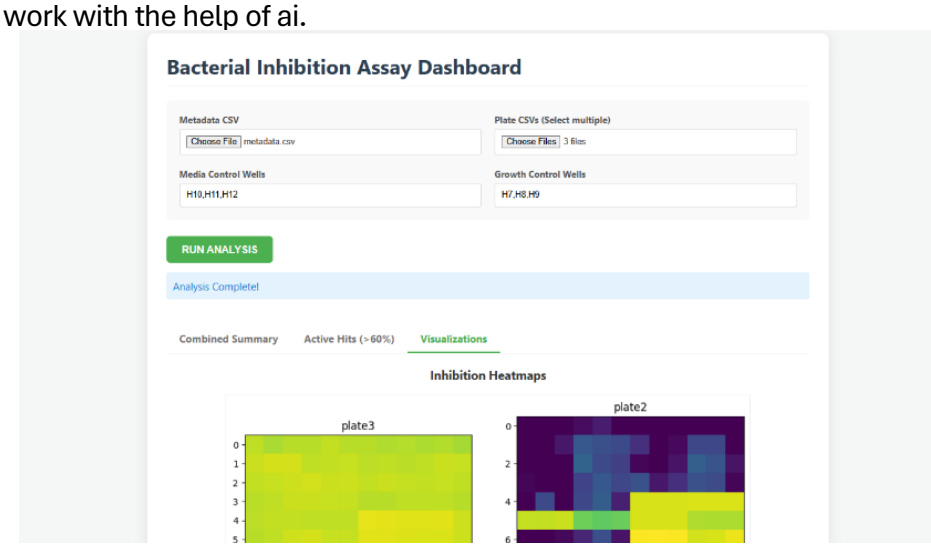


Figure 1. Bioactivity profiles of 14 HPLC fractions against various pathogens. The bar graph shows mean % inhibition of 14 HPLC fractions against Gram-positive and Gram-negative bacteria. Each plate corresponds to different pathogen (Plate 1-*Acinetobacter baumannii*, Plate 2 – *Pseudomonas aeruginosa*, Plate 3 – *Staphylococcus aureus*, and Plate 4 – *Vibrio* sp). Positive values indicate bacterial inhibition relative to controls, while negative values indicate apparent growth stimulation under the tested conditions. Fraction 1–6 showed strong broad-spectrum activity (>70%), while all fractions inhibited *Staphylococcus aureus* (>60%). Selective activity was observed for fractions 1–13 and 1–14 against *Pseudomonas aeruginosa*. Error bars represent standard error (SE) from three biological replicates, indicating variability in the resazurin-based microdilution assay.



Note: Each plate represents a different pathogen.1- A. baumannii, 2- P. aeruginosa, 3- S. aureus, 4- Vibrio sp. D represents doses in ug/mL

Figure 2. Heatmap of bioactivity profiles of 14 HPLC fractions against bacterial pathogens. Each tile represents the mean % inhibition of 14 HPLC fractions across multiple bacterial pathogens, representing both Gram-positive and Gram-negative species. Fraction 1–6 exhibited consistently high bioactivity (>70%) across all tested pathogens, indicating broad-spectrum antimicrobial activity. Notably, all fractions (1-1 to 1-14) demonstrated strong inhibitory effects (>60%) against Gram-positive *Staphylococcus aureus*. Selective activity was also observed for fractions 1–13 and 1–14 against *Pseudomonas aeruginosa*, suggesting pathogen-specific sensitivity. Each tile represents the mean % inhibition derived from three biological replicates using a resazurin-based microdilution assay. Data processing, normalization, and visualization were performed using an automated Python pipeline.

################

Additionally, this pipeline was shared to my former lab mates in the Philippines who also performed the same assay. One colleague who was very good at coding did create a GUI out of this work with the help of ai.